

Midterm Exam – Objective Questions

1. Which decorator is used to define a GET endpoint in FastAPI?
 - ☐ A. `@app.route()`
 - ☒ B. `@app.get()`
 - ☐ C. `@app.fetch()`
 - ☐ D. `@get.route()`
2. What is the default documentation URL for a FastAPI application?
 - ☐ A. `/home`
 - ☒ B. `/docs`
 - ☐ C. `/apidoc`
 - ☐ D. `/swagger`
3. What HTTP method should be used to update an existing resource?
 - ☐ A. GET
 - ☐ B. DELETE
 - ☒ C. PUT
 - ☐ D. POST
4. Which of the following HTTP methods is used to delete a resource?
 - ☐ A. `@app.post()`
 - ☐ B. `@app.remove()`
 - ☐ C. `@app.get()`
 - ☒ D. `@app.delete()`
5. FastAPI is built on top of which Python web framework?
 - ☐ A. Flask
 - ☐ B. Tornado
 - ☒ C. Starlette
 - ☐ D. Django

6. What is returned when FastAPI route functions return a dictionary?
- ☐ A. HTML
 - ☒ **B. JSON**
 - ☐ C. CSV
 - ☐ D. XML
7. Which of the following methods is used to create new data in a RESTful API?
- ☐ A. PUT
 - ☒ **B. POST**
 - ☐ C. DELETE
 - ☐ D. GET
8. In FastAPI, what type of data structure is automatically validated and serialized?
- ☐ A. Dict
 - ☐ B. Class
 - ☒ **C. Pydantic Model**
 - ☐ D. Function
9. How can you start a FastAPI server using uvicorn?
- ☐ A. python app.py
 - ☐ B. flask run
 - ☒ **C. uvicorn main:app --reload**
 - ☐ D. run main.py
10. Which method handles route dependencies in FastAPI?
- ☐ A. Include()
 - ☐ B. Requires()
 - ☒ **C. Depends()**
 - ☐ D. Attach()

11. What is the main purpose of using Depends in FastAPI?

- A. Validate data
- B. Import modules
- **C. Inject dependencies**
- D. Connect to the internet

12. Which module provides the Depends function?

- A. typing
- **B. fastapi**
- C. pydantic
- D. requests

13. What will the following return?

```
def get_token():  
    return {"token": "abc123"}
```

```
@app.get("/auth")
```

```
def auth(data = Depends(get_token)):  
    return data
```

- A. "abc123"
- B. None
- **C. {"token": "abc123"}**
- D. Error

14. Dependency functions can be used to:

- A. Format JSON
- **B. Simulate databases or authentication**
- C. Create lists
- D. Declare loops

15. Dependencies can be injected into:

- A. Only POST routes
- B. Only global middleware
- **C. Any route function**
- D. Only classes

16. What is the correct annotation for a variable age of type integer?

- A. age -> int
- B. int age
- **C. age: int**
- D. age := int

17. What is the type annotation for a list of strings?

- A. List<string>
- B. list[str]
- **C. List[str]**
- D. Str[List]

18. Which module is required to use List, Dict, and Tuple type hints?

- **A. typing**
- B. sys
- C. types
- D. utils

19. How do you annotate a dictionary with string keys and integer values?

- A. Dict<string, int>
- B. dict[str: int]
- **C. Dict[str, int]**
- D. Dictionary[str, int]

20. What is the correct way to annotate a function returning a float?

- A. `def fun(): float`
- **B. `def fun() -> float:`**
- C. `def fun: float`
- D. `def fun() := float`

21. Which is the correct annotation for a tuple of (str, int)?

- A. `Tuple<str, int>`
- B. `(str, int)`
- **C. `Tuple[str, int]`**
- D. `List[str, int]`

22. `Set[int]` defines a:

- A. Group of unordered strings
- **B. Unique set of integers**
- C. Duplicate list of integers
- D. Dictionary of ints

23. The annotation `Optional[int]` means:

- A. The variable can be a string or integer
- B. The variable must always be an integer
- **C. The variable can be int or None**
- D. The variable is not required at all

24. Which is a correct usage of Union?

- A. `Union[int, float]`
- B. `int | float` (Python 3.10+)
- **C. Both A and B**
- D. Neither A nor B

25. What does `Optional[str]` resolve to?

- **A. `Union[str, None]`**
- B. `List[str]`
- C. `Tuple[str]`
- D. `None[str]`

26. Pydantic models are subclasses of:

- A. Schema
- **B. `BaseModel`**
- C. Dataclass
- D. Validator

27. What is the purpose of Pydantic in FastAPI?

- A. Formatting
- B. Logging
- **C. Data validation**
- D. Authentication

28. What will this code do?

```
class User(BaseModel):
```

```
    name: str
```

```
    age: int
```

- **A. Validate the user input**
- B. Log the input
- C. Create a form
- D. Create a tuple

29. Which keyword is used to define nested Pydantic models?

- A. `embed`
- B. `model`
- C. `class`
- **D. `List[ModelName]`**

30. What is the output type of a Pydantic model?

- A. HTML
- **B. JSON**
- C. XML
- D. YAML

31. What happens if you provide invalid data to a Pydantic model?

- A. Accepts it
- B. Ignores it
- **C. Raises validation error**
- D. Stores None

32. Nested Pydantic models can be used to represent:

- A. Variables
- **B. Relational data structures**
- C. Functions
- D. Endpoints

33. Which of the following imports is required for Pydantic?

- A. import dataclasses
- B. from fastapi import Pydantic
- **C. from pydantic import BaseModel**
- D. import typing

34. How do you annotate a list of Pydantic models?

- **A. List[ModelName]**
- B. [ModelName]
- C. ModelName[]
- D. List<ModelName>

35. Which of the following Pydantic field types is optional?

- **A. Optional[str]**
 - B. List[str]
 - C. Tuple[str, int]
 - D. Dict[str, int]
-

36. Which import is necessary for using List, Optional, Union?

- A. import list
- B. from pydantic import List
- **C. from typing import List, Optional, Union**
- D. from fastapi.typing import *

37. What will the following function return?

```
def add(a: int, b: int) -> int:
```

```
    return a + b
```

- A. Any value
- B. A string
- **C. An integer**
- D. None

38. Identify the error:

```
class Department(BaseModel):
```

```
    name: str
```

```
    employees: List[Employee]
```

- **A. Employee must be defined before use**
- B. List is invalid
- C. BaseModel must be imported from typing
- D. None of the above

39. What is the correct way to define an endpoint that returns a list of employees?

- **A. `def get_employees() -> List[str]:`**
- B. `def get_employees: List[str]`
- C. `@app.get("/employees") -> str`
- D. `def get_employees() = List[str]`

40. How does FastAPI convert Pydantic models into API documentation?

- A. Using Flask templates
- B. Using a decorator
- **C. Using OpenAPI and Swagger UI**
- D. It doesn't convert them

WRAP

Question 1:

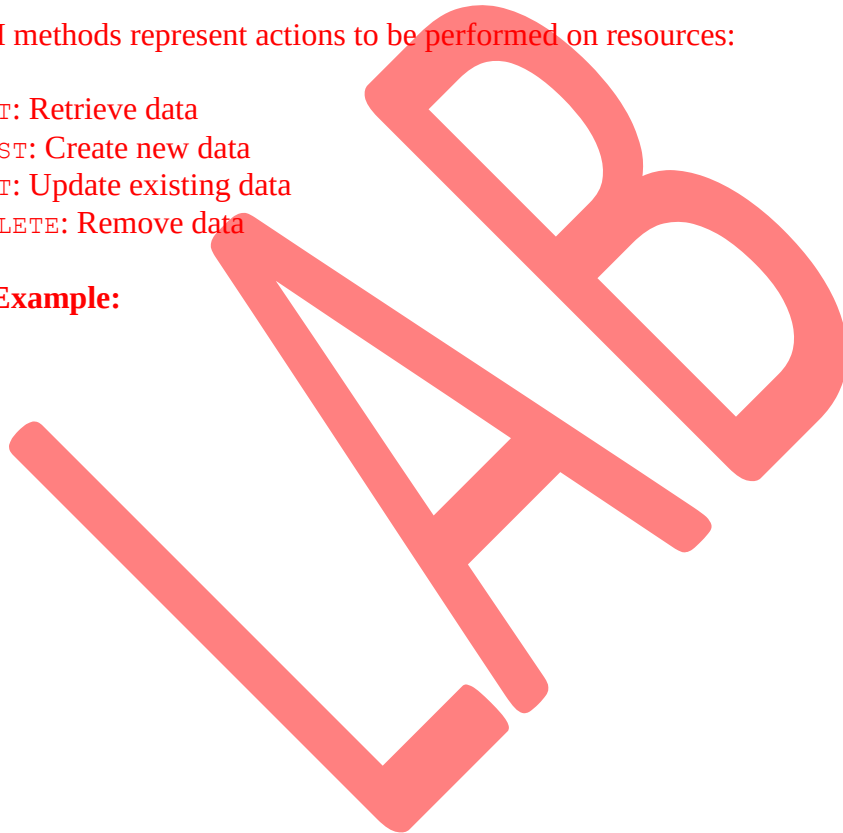
Explain how REST API methods (GET, POST, PUT, DELETE) are implemented in FastAPI. Create a simple FastAPI app for managing a list of books where users can:

- View all books
- Add a new book
- Update an existing book
- Delete a book

Answer:

REST API methods represent actions to be performed on resources:

- GET: Retrieve data
- POST: Create new data
- PUT: Update existing data
- DELETE: Remove data

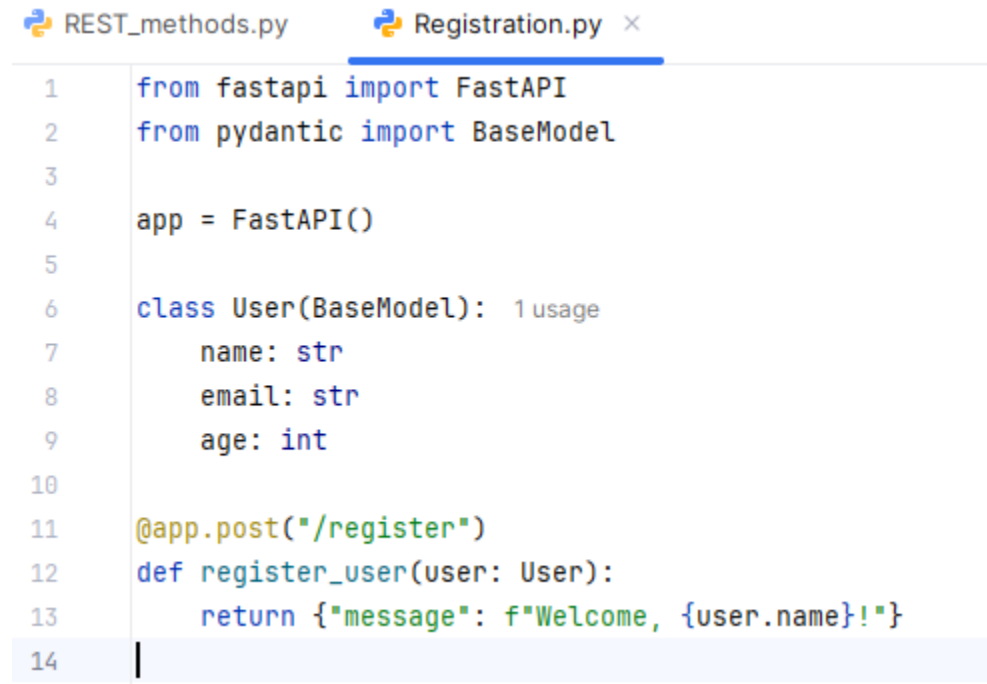
FastAPI Example:

REST_methods.py

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 from typing import List
4
5 app = FastAPI()
6
7 class Book(BaseModel): 3 usages
8     id: int
9     title: str
10    author: str
11
12 books: List[Book] = []
13
14 @app.get("/books")
15 def get_books():
16     return books
17
18 @app.post("/books")
19 def add_book(book: Book):
20     books.append(book)
21     return {"message": "Book added successfully"}
22
23 @app.put("/books/{book_id}")
24 def update_book(book_id: int, updated_book: Book):
25     for i, book in enumerate(books):
26         if book.id == book_id:
27             books[i] = updated_book
28             return {"message": "Book updated"}
29     return {"error": "Book not found"}
30
31 @app.delete("/books/{book_id}")
32 def delete_book(book_id: int):
33     global books
34     books = [book for book in books if book.id != book_id]
35     return {"message": "Book deleted"}
36
```

Question 2:

Write a FastAPI route that receives user registration data using a Pydantic model (name, email, age) and returns a success message with the user's name. Also explain how FastAPI automatically validates and serializes the input data.



The screenshot shows a code editor with two tabs: 'REST_methods.py' and 'Registration.py'. The 'Registration.py' tab is active and contains the following Python code:

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3
4 app = FastAPI()
5
6 class User(BaseModel): 1 usage
7     name: str
8     email: str
9     age: int
10
11 @app.post("/register")
12 def register_user(user: User):
13     return {"message": f"Welcome, {user.name}!"}
14
```

Explanation:

- The `User` model enforces correct types.
- FastAPI parses the request body into the model and raises a validation error if types are incorrect

◆ Section B: Dependency Injection

Question 3:

Define Dependency Injection in FastAPI. Write a practical example where a dependency function is used to simulate user authentication by checking for a token in the header. Use Depends in your route to access the token.

Answer:

Dependency Injection allows for reusable logic like authentication.

```
REST_methods.py × Registration.py Dependency.py ×
1 from fastapi import Depends, HTTPException, Header
2
3 def get_token(token: str = Header(...)): 1 usage
4     if token != "validtoken123":
5         raise HTTPException(status_code=401, detail="Invalid token")
6     return token
7
8 @app.get("/secure-data")
9 def read_secure_data(token: str = Depends(get_token)):
10     return {"message": "Access granted", "token": token}
11
```

Explanation:

- `Depends(get_token)` injects the token dependency.
- Ensures endpoints requiring auth all use centralized logic.

Question 4:

Create a reusable dependency that connects to a mock database (as a Python dictionary) and use it in multiple endpoints (e.g., /users, /orders) to simulate a shared resource. Explain the benefits of using Depends.

```
REST_methods.py  Registration.py  Dependency.py  DEP.py ×
1  from fastapi import Depends
2
3  def get_mock_db(): 2 usages
4      return {"users": ["Alice", "Bob"], "orders": ["#123", "#124"]}
5
6  @app.get("/users")
7  def read_users(db = Depends(get_mock_db)):
8      return db["users"]
9
10 @app.get("/orders")
11 def read_orders(db = Depends(get_mock_db)):
12     return db["orders"]
13
```

Explanation:

- The `get_mock_db` function simulates a reusable database connection.

Question 5:

Explain the importance of type annotations in Python. Using code examples, annotate the following:

- A list of strings
- A tuple of (int, str)
- A dictionary with string keys and float values
- A function that accepts an int and float, and returns a str

```
ST_methods.py × Registration.py Dependency.py DEP.py Type_Anotation.py × ⌵ ⋮
1 from typing import List, Tuple, Dict
2
3 names: List[str] = ["Alice", "Bob"]
4 person: Tuple[int, str] = (1, "John")
5 scores: Dict[str, float] = {"Math": 88.5}
6
7 def greet_user(user_id: int, balance: float) -> str:
8     return f"User {user_id} has ${balance}"
9 |
```

Question 6:

Write a function that takes an optional discount code (str) and a price (float). If the discount is provided, reduce the price by 10%. Use Optional in your function annotation and explain how it helps in flexible API input handling.

```
.py Dependency.py DEP.py Type_Anotation.py Optional_in_Functions.py × ⌵
1 from typing import Optional
2
3 def calculate_price(price: float, discount_code: Optional[str] = None) -> float:
4     if discount_code:
5         return price * 0.9
6     return price
7
```

Question 7:

Differentiate between Union and Optional in Python type annotations. Create a Pydantic model Product that accepts either a string or an integer as the id, and optionally includes a description.

```
DEP.py  Type_Annotation.py  Optional_in_Functions.py  Union_vs_Optional.py x v ⋮
1  from pydantic import BaseModel
2  from typing import Union, Optional
3
4  class Product(BaseModel):
5      id: Union[int, str]
6      description: Optional[str] = None
7  |
```

Explanation:

- `Union[int, str]`: id can be either.
- `Optional[str]`: description may be missing.

Question 8:

Write a FastAPI route using a Pydantic model where a user can submit either a phone number (int) or an email (str) as contact info using Union. Explain how this is helpful in flexible API designs.

```
Type_Annotation.py  Optional_in_Functions.py  Union_vs_Optional.py  Contact_info.py x v
1  from fastapi import FastAPI
2  from pydantic import BaseModel
3  from typing import Union
4
5  app = FastAPI()
6
7  class Contact(BaseModel): 1 usage
8      contact_info: Union[int, str] # phone or email
9
10 @app.post("/contact")
11 def submit_contact(contact: Contact):
12     return {"contact": contact.contact_info}
13  |
```

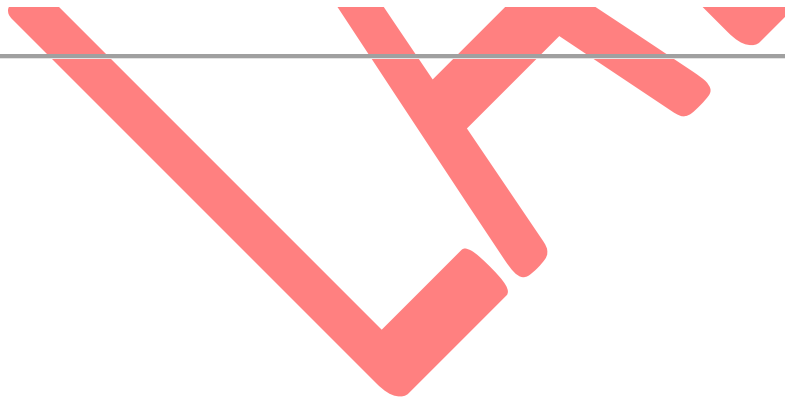
◆ Section E: Pydantic & Nested Models

Question 9:

Create two Pydantic models: Employee (with name, role) and Department (with name, employees: List[Employee]). Write a FastAPI route that accepts a Department object and returns the total number of employees.

Optional_in_Functions.py Union_vs_Optional.py Contact_info.py **Nested_pydantic.py** × ▾

```
1  ▾ from typing import List
2  from pydantic import BaseModel
3
4  ▾ class Employee(BaseModel): 1 usage
5      name: str
6      role: str
7
8  ▾ class Department(BaseModel): 1 usage
9      name: str
10     employees: List[Employee]
11
12     @app.post("/department")
13     ▾ def department_info(dept: Department):
14         return {"department": dept.name, "employee_count": len(dept.employees)}
15
```



Question 10:

Explain how Pydantic helps in data validation and serialization in FastAPI. Write a nested Pydantic schema for a school system with School, Class, and Student models. Show how FastAPI uses this model to accept nested JSON input.

```
Type_Anotation.py Optional_in_Functions.py Union_vs_Optional.py Contact_info.py Nested_pydantic.py NEST_models.py ×  
from pydantic import BaseModel  
from typing import List  
  
class Student(BaseModel): 1 usage  
    name: str  
    grade: int  
  
class Class(BaseModel): 1 usage  
    class_name: str  
    students: List[Student]  
  
class School(BaseModel): 1 usage  
    school_name: str  
    classes: List[Class]  
  
@app.post("/school")  
def receive_school_data(school: School):  
    total_students = sum(len(cls.students) for cls in school.classes)  
    return {"school": school.school_name, "total_students": total_students}
```

Explanation:

- Validates nested data automatically.
- Easy to traverse structured input for processing

Common HTTP Status Codes in Responses:

Code	Meaning	Description
200	OK	Request was successful
201	Created	Resource was created (usually after POST)
400	Bad Request	Client sent an invalid request
401	Unauthorized	Authentication required or failed
404	Not Found	Requested resource doesn't exist
500	Internal Server Error	Something went wrong on the server

CLASS WORK

1. A student's name (text)

- The student's name is a **string**.

python

Copy code

```
student_name: str = "Mohamed"
```

2. A list of exam scores (numbers)

- The exam scores are **integers**, and since they are in a list, we annotate the list to contain integers.

python

Copy code

```
exam_scores: List[int] = [85, 92, 78, 90]
```

3. A dictionary of country populations (key: country name, value: population number)

- The dictionary has **string keys** (country names) and **integer values** (population numbers).

python

Copy code

```
from typing import Dict
```

```
country_populations: Dict[str, int] = {"China": 1412000000, "India": 1380000000}
```

4. A tuple representing a date (day, month, year)

- The tuple consists of:
 - An **integer** (day),
 - A **string** (month),
 - An **integer** (year).

```
python
```

```
Copy code
```

```
independence_day: Tuple[int, str, int] = (4, "July", 1776)
```

5. A variable that could be either an integer or floating-point number

- This variable could be either an **integer** or a **float**. So, we use **Union** to specify that it could be either.

```
python
```

```
Copy code
```

```
from typing import Union
```

```
measurement: Union[int, float] = 3.5
```

Final Annotated Code:

```
python
```

```
Copy code
```

```
from typing import List, Dict, Tuple, Union
```

```
student_name: str = "Mohamed"
```

```
exam_scores: List[int] = [85, 92, 78, 90]
```

```
country_populations: Dict[str, int] = {"China": 1412000000, "India": 1380000000}
```

```
independence_day: Tuple[int, str, int] = (4, "July", 1776)
```

```
measurement: Union[int, float] = 3.5
```

1. This list should only contain strings

- The list names should contain **strings**, not integers.

```
python
```

Copy code

```
names: List[str] = ["Alice", "Bob", "Charlie"]
```

2. This dictionary should map strings to booleans

- The dictionary flags should map **strings** to **booleans**, not **strings** to **strings**.

```
python
```

Copy code

```
flags: Dict[str, bool] = {"is_active": True, "is_admin": False}
```

3. This variable should accept numbers or None

- The variable price should accept either an **integer** or **None** (not str).

```
python
```

Copy code

```
price: Union[int, None] = None
```

4. This tuple should have exactly 3 elements: str, int, bool

- The tuple person should have 3 elements:
 - The first element should be a **string** (str),
 - The second element should be an **integer** (int),
 - The third element should be a **boolean** (bool).
- The current tuple is incorrect because the second element is a string instead of an integer.

python

Copy code

```
person: Tuple[str, int, bool] = ("Ali", 25, True)
```

5. This set should contain only integers

- The set unique_values should contain **integers**, not strings.

python

Copy code

```
unique_values: Set[int] = {1, 5, 10, 10}
```

Final Corrected Code:

python

Copy code

```
from typing import List, Dict, Union, Tuple, Set
```

1. This list should only contain strings

```
names: List[str] = ["Alice", "Bob", "Charlie"]
```

2. This dictionary should map strings to booleans

```
flags: Dict[str, bool] = {"is_active": True, "is_admin": False}
```

3. This variable should accept numbers or None

```
price: Union[int, None] = None
```

4. This tuple should have exactly 3 elements: str, int, bool

```
person: Tuple[str, int, bool] = ("Ali", 25, True)
```

5. This set should contain only integers

```
unique_values: Set[int] = {1, 5, 10, 10}
```

1. Book Inventory

For the book inventory, the fields are:

- Title: a string,
- Price: a float,
- Available formats: a tuple of strings,
- In stock?: a boolean,
- Number of pages: an optional integer (it may or may not be present).

python

Copy code

```
from typing import Optional, Tuple
```

```
# Book Inventory
```

```
book_inventory: Dict[str, Union[str, float, Tuple[str, ...], bool, Optional[int]]] = {
```

```
    "title": "Python Programming 101", # string
```

```
    "price": 29.99, # float
```

```
    "available_formats": ("eBook", "Paperback", "Hardcover"), # tuple of strings
```

```
    "in_stock": True, # boolean
```

```
    "number_of_pages": 350 # Optional integer
```

}

- **Tuple[str, ...]** indicates a tuple of strings of any length.
 - **Optional[int]** means `number_of_pages` could be an int or None.
-

2. Social Media Post

For the social media post, the fields are:

- Content: a string,
- Likes: an integer,
- Tags: a list of strings,
- Pinned status: a boolean,
- Location: a string or None (the location might not always be available).

python

Copy code

```
from typing import List, Optional
```

```
# Social Media Post
```

```
social_media_post: Dict[str, Union[str, int, List[str], bool, Optional[str]]] = {  
    "content": "Learning Python is fun!",    # string  
    "likes": 1200,                          # integer  
    "tags": ["Python", "Programming", "Tutorial"], # list of strings  
    "pinned_status": True,                  # boolean  
    "location": None                       # Optional string (could be None)  
}
```

- **List[str]** is used for the tags field to indicate that it's a list of strings.
 - **Optional[str]** means location could be a string or None.
-

3. Scientific Measurement

For the scientific measurements, the list can contain:

- Temperature readings: float,
- Sensor IDs: string,
- Error codes: integer,
- Validity flags: boolean.

python

Copy code

```
from typing import List, Union
```

```
# Scientific Measurement
```

```
scientific_measurements: List[Union[float, str, int, bool]] = [
```

```
    25.4,      # temperature reading (float)
```

```
    "SensorA001", # sensor ID (string)
```

```
    404,      # error code (integer)
```

```
    True      # validity flag (boolean)
```

```
]
```

- **List[Union[float, str, int, bool]]** is used for the list because it can contain any of these types in any order.

Summary of Final Code

python

Copy code

```
from typing import List, Dict, Union, Optional, Tuple
```

```
# 1. Book Inventory
```

```
book_inventory: Dict[str, Union[str, float, Tuple[str, ...], bool, Optional[int]]] = {
```

```
    "title": "Python Programming 101",
```

```
"price": 29.99,  
"available_formats": ("eBook", "Paperback", "Hardcover"),  
"in_stock": True,  
"number_of_pages": 350  
}
```

2. Social Media Post

```
social_media_post: Dict[str, Union[str, int, List[str], bool, Optional[str]]] = {  
    "content": "Learning Python is fun!",  
    "likes": 1200,  
    "tags": ["Python", "Programming", "Tutorial"],  
    "pinned_status": True,  
    "location": None  
}
```

3. Scientific Measurement

```
scientific_measurements: List[Union[float, str, int, bool]] = [  
    25.4,      # temperature reading  
    "SensorA001", # sensor ID  
    404,      # error code  
    True      # validity flag  
]
```